

A Query Recommender for Preference-Aware Natural Language Querying

Implementation Report — Phase 3 (Behavioral Preference Profile)

The “stated vs. actual preference” signal for the query recommender

Narges

Supervisor: Prof. Elisa Quintarelli

July 28, 2026

Abstract

This report documents Phase 3 of the thesis: the construction of a per-user **behavioral preference profile** on the Movie domain, contrasting what a user *says* they want (inferred from free-text search queries) against what they *actually* engage with (inferred from weighted watch behavior). This is the empirical substrate for the thesis’s central premise — that stated and real preferences diverge, and that a query recommender must condition on both to be useful. Unlike Phase 0/1 (data/infrastructure preparation and a development harness, reported separately and not thesis deliverables in their own right), Phase 3 **is** part of the thesis’s actual contribution: it produces the behavioral divergence signal that Phase 4’s candidate-generation step will consume. This report first gives a high-level overview of the pipeline that produces this signal, then explains each part of it in detail, and reports the verification results obtained on the full `netflix_dataset`.

Contents

1	Introduction	3
1.1	Scope	3
1.2	Where this fits	3
2	High-Level Overview	4
2.1	The core idea	4
2.2	Pipeline diagram	4
3	Detailed Walkthrough	6
3.1	Inputs	6
3.2	Catalog vocabulary and search-query tagging	6
3.3	Actual-interest weighting	6
3.4	Search→watch temporal join	7
3.5	Stated-interest weighting	7
3.6	Divergence scoring	7
3.7	Output writing	7
4	Output Schema	8
5	Verification and Results	8
6	Interpretation and Limitations	9

1 Introduction

The professor’s scoping email attributes to “Jerry’s” analysis the observation that a user’s *stated* queries and their *real* interaction behavior often diverge — a user may type “*romantic movie*” but consistently watch American comedies. A query recommender that only looks at query history is, per this framing, little more than autocomplete; a genuinely useful one must condition on both signals. Phase 3 builds the first half of that requirement: a machine-readable, per-user summary of what a user says they want versus what they actually engage with, quantified as a single divergence score.

1.1 Scope

Phase 3 operates on the **Movie domain only**. As established during Phase 0, the Tourism domain’s only large behavioral table (`log_vc`, VeronaCard tap-in visits) is keyed to an anonymous card, not a named user, and was explicitly excluded from the ontology — so Tourism has no per-user behavioral signal to condition on. All behavior-aware work in this thesis is therefore carried by the Movie domain’s `netflix_dataset/` (a synthetic, Faker-generated dataset with 10,300 users, 1,040 movies, 105,000 watch events, and 26,500 search-log entries).

1.2 Where this fits

Phase 3 is implemented as a standalone script, `recommender/behavioral_profile.py`, in a new top-level `recommender/` directory — the start of the actual thesis deliverable, per the scope decision recorded in the companion *Phase 0 & Phase 1 Implementation Report*. It is kept separate from `profile_geneartion/` (an earlier, more general interest-profile tool from the system’s original architecture), which remains a reference for the interest-node JSON shape but is not extended in place. Phase 3’s output — one JSON profile per user — is the direct input to Phase 4’s recommendation-candidate generation step, described in the project roadmap.

2 High-Level Overview

2.1 The core idea

For each user, Phase 3 computes two independent rankings over the same vocabulary of catalog categories (genre, content type, country of origin): one derived from what they *searched for*, one from what they *watched*. It then measures how much the two rankings agree. Three design choices make this measurement meaningful rather than a raw count comparison:

- (a) **Weighting, not counting.** A watch event that a user completed counts more than one they abandoned after 5%; this is implemented as a continuous *completion weight* rather than a binary “watched / not watched” signal (Section 3.3).
- (b) **An engineered link between the two data sources.** `search_logs` and `watch_history` share no foreign key in the source dataset — there is no column recording which movie, if any, a search led to. A temporal join (nearest subsequent watch within a time window) is engineered to recover this link where it plausibly exists (Section 3.4), producing a concrete artifact of individual search→watch events, independent of the aggregate divergence score.
- (c) **A single summary statistic.** The two rankings’ top-5 categories are compared via Jaccard similarity, giving one number per user that later phases (and this report’s results) can reason about directly (Section 3.6).

2.2 Pipeline diagram

Figure 1 shows the full data flow, from the three source CSVs to the two output artifacts. The left branch (green) computes the *actual* signal from watch behavior; the right branch (orange) computes the *stated* signal from search behavior. Both converge on a per-user divergence score and a combined JSON profile.

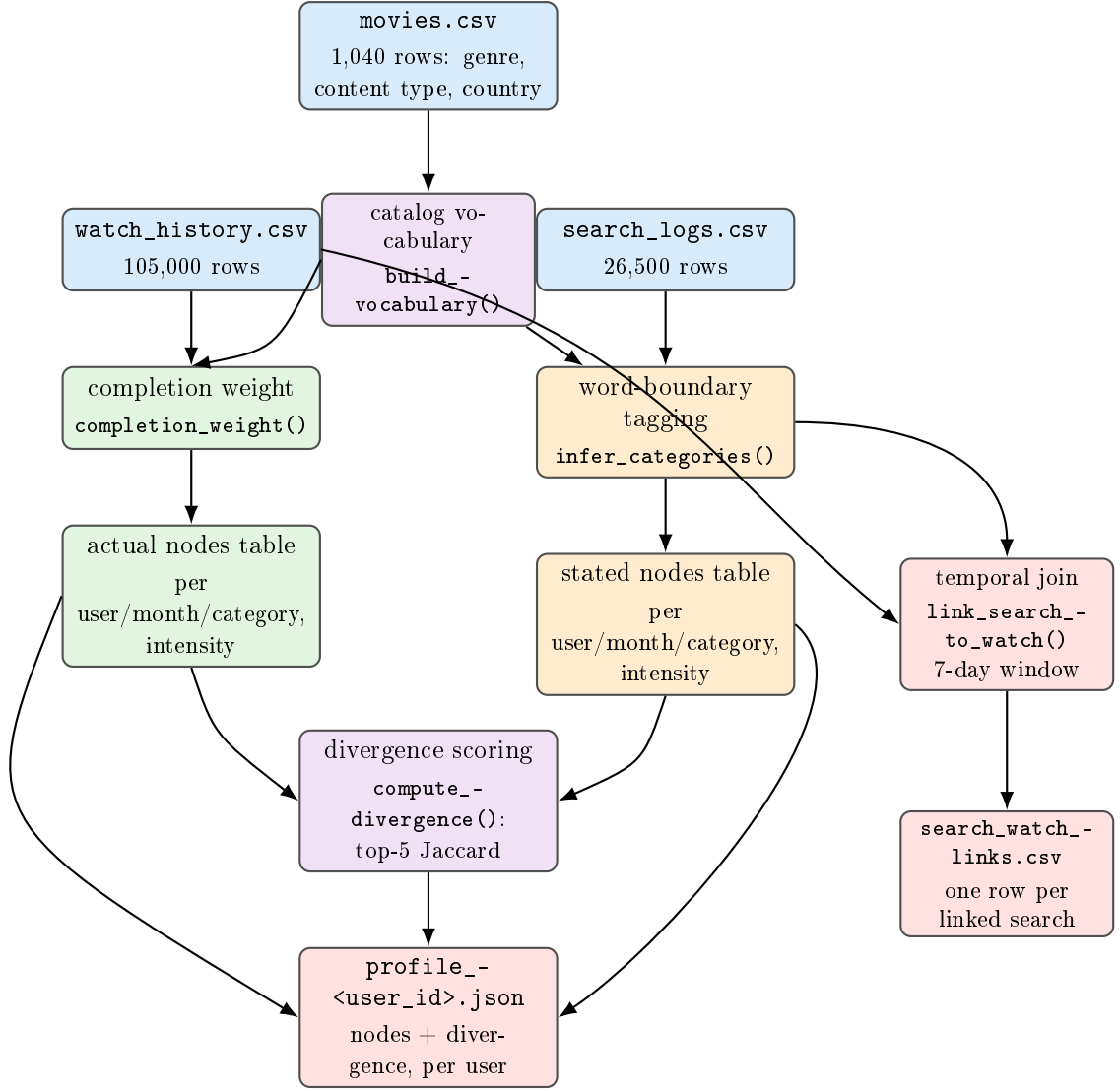


Figure 1: Phase 3 data flow. The catalog vocabulary (top) is used both to tag search queries and, implicitly, to define the category space watch events are grouped into. The actual (green) and stated (orange) branches each produce a per-user/month/category intensity table; these feed both the per-user JSON profiles (with all nodes plus a divergence summary) and, independently, a divergence score. The temporal join (bottom left, red) produces a separate, row-level output.

3 Detailed Walkthrough

This section explains each stage of `recommender/behavioral_profile.py` (244 lines) in the order it executes, from `main()`.

3.1 Inputs

`load_data()` reads the three source CSVs and left-joins `watch_history` against `movies` on `movie_id` to attach each watch event's genre/content-type/country. Rows with a missing `user_id` or `movie_id` are dropped. Missing category values in the catalog are filled with the literal string "Unknown" so downstream groupings do not silently drop rows with incomplete metadata.

3.2 Catalog vocabulary and search-query tagging

`build_vocabulary()` collects every distinct, non-“Unknown” value across `genre_primary`, `genre_secondary`, `content_type`, and `country_of_origin`, sorted **longest string first**. This ordering matters: it guarantees a multi-word value such as "Stand-up Comedy" is checked before a shorter value like "Comedy" could match part of it.

`infer_categories()` then tags each free-text `search_query` with every vocabulary value it contains, using a **word-boundary regular expression** (`\bvalue\b`), not a plain substring test. This is a deliberate bug fix made during implementation:

Listing 1: The word-boundary tagging function. An earlier, substring-based version let `content_type="Movie"` match inside the plural word “movies” in nearly every query, and would similarly have let "War" match inside “warrior.”

```
def build_vocabulary_patterns(vocabulary):
    # word-boundary match: plain substring would let e.g. "Movie" (a content_type
    # value) spuriously match inside the plural "movies", or "War" match inside
    # "warrior"
    return {v: re.compile(r"\b" + re.escape(v.lower()) + r"\b") for v in vocabulary}

def infer_categories(search_query, vocabulary_patterns):
    query = str(search_query).lower()
    return [v for v, pattern in vocabulary_patterns.items() if pattern.search(query)]
```

3.3 Actual-interest weighting

`completion_weight()` converts each `watch_history` row into a weight in $[0.05, 1.0]$:

$$w(\text{row}) = \begin{cases} 1.0 & \text{if action = "completed"} \\ \text{clip}\left(\frac{\text{progress_percentage}}{100}, 0.05, 1.0\right) & \text{if progress is known} \\ 0.3 & \text{otherwise} \end{cases}$$

This operationalizes “actual interest” as *how much* of something a user engaged with, not merely whether a watch event was logged — a user who starts and abandons five thrillers should not count for as much as one who finishes a single thriller.

`compute_actual_nodes_table()` groups weighted watches by (`user_id`, `month`, `category-value`), divides by that user-month's total weighted interactions to obtain a relative **intensity** in $[0, 1]$ (rounded to two decimals, the same relative-share convention used by the earlier `profile-geneartion` interest-node model), and labels every row `signal = "actual"`.

3.4 Search→watch temporal join

`link_search_to_watch()` addresses a gap identified during data inventory: there is no existing key linking a `search_logs` row to a specific movie in `watch_history`. For every search with at least one inferred category (Section 3.2), the function finds that user’s **next watch within a 7-day window**, implemented with `pandas`’ `merge_asof` (forward direction, 7-day tolerance, grouped by=`"user_id"`):

Listing 2: The temporal join. `matched_stated_intent` flags whether any category inferred from the search actually appears among the linked watch’s own genre/content-type — a row-level, individually inspectable artifact of stated-vs-actual correspondence, independent of the aggregate divergence score in Section 3.6.

```
linked = pd.merge_asof(
    searches, watch_sorted,
    left_on="search_date", right_on="watch_date", by="user_id",
    direction="forward", tolerance=pd.Timedelta(days=7),
    suffixes=("", "_watched"),
)
linked = linked.dropna(subset=["movie_id"]).copy()
linked["matched_stated_intent"] = linked.apply(
    lambda r: bool(set(r["inferred_categories"])
                     & {r["genre_primary"], r["genre_secondary"], r["content_type"]}),
    axis=1,
)
```

Rows without a watch inside the window are dropped, so the resulting table (written to `search_watch_links.csv`) contains only searches that were plausibly followed by a corresponding viewing session.

3.5 Stated-interest weighting

`compute_stated_nodes_table()` mirrors Section 3.3’s computation for the stated side: each search’s inferred categories are exploded into one row per category, grouped by (`user_id`, `month`, `category`), and normalized by that user-month’s total tagged-search count to give an intensity in $[0, 1]$. Rows are labeled `signal = "stated"`.

3.6 Divergence scoring

`compute_divergence()` takes each user’s **top-5** actual categories and top-5 stated categories, ranked by summed intensity across all months, and computes:

- **overlap** — the set intersection of the two top-5 lists;
- **jaccard_similarity** — $|A \cap S| / |A \cup S|$ for actual set A and stated set S (`None` if a user has neither signal at all).

This single number is the behavioral divergence signal Phase 4’s candidate generation and ranking will condition on.

3.7 Output writing

`write_user_profiles()` concatenates the actual and stated node tables, groups by `user_id`, and writes one JSON file per user to `recommender/output/behavioral_profiles/profile_<user_id>.json`, combining every node (actual and stated, across all months) with that user’s single divergence summary. The full schema is given in Section 4.

4 Output Schema

Listing 3: `profile_<user_id>.json` schema.

```
{
  "user_id": "user_00001",
  "domain": "Movie",
  "nodes": [
    {
      "category": "Genre | Content Type | Country | Stated Interest",
      "value": "<category value>",
      "signal": "actual | stated",
      "intensity": 0.0-1.0,
      "time_window": { "type": "monthly", "value": "YYYY-MM" },
      "support": { "weighted_interactions": <float> }
        | { "searches": <int> }
    }
  ],
  "divergence": {
    "actual_top": ["<top-5 actual categories by intensity>"],
    "stated_top": ["<top-5 stated categories by intensity>"],
    "overlap": ["<categories in both>"],
    "jaccard_similarity": 0.0-1.0
  }
}
```

This is structurally analogous to the earlier `profile_geneartion` tool’s `nodi_interesse` shape (the same category/value/intensity/time-window fields), extended with the `signal` field distinguishing stated from actual, and the top-level `divergence` block summarizing the gap between them.

5 Verification and Results

The script was run against the full `netflix_dataset` (1,040 movies, 10,300 users, 105,000 watch rows, 26,500 search rows).

Metric	Value
Users with a behavioral profile written	10,000 of 10,300
Search→watch links found (7-day window)	1,179
Average stated-vs-actual Jaccard similarity	0.01
Users with zero overlap (top-5 stated vs. top-5 actual)	9,399 / 10,000

Table 1: Phase 3 verification results on the full dataset. 300 users have no watch history and therefore receive no profile.

As a spot check, `profile_user_00001.json` was inspected directly: its actual top-5 was `["Movie", "USA", "Canada", "Adventure", "War"]` (a mix of content-type, country, and genre values, since all three category types share one ranking pool), its stated top-5 was `["Action"]` (this user had only one tagged search in the log), the overlap was empty, and the Jaccard similarity was 0.0 — consistent with the aggregate numbers above.

The word-boundary fix (Section 3.2) was confirmed to be working: searches containing the plural word “movies” no longer spuriously tag `content_type="Movie"`, and “warrior”-type queries no longer spuriously tag `genre="War"`.

6 Interpretation and Limitations

- **The near-zero average Jaccard similarity (0.01) is a striking number**, and its *direction* is consistent with the professor’s own framing example (a user types “romantic movie” but consistently watches American comedies). It should **not**, however, be reported as a validated real-world divergence rate. `netflix_dataset` is Faker-generated synthetic data with **no true causal link** between a search string and a later watch event — a search and a subsequent watch being uncorrelated is exactly what random generation would produce, so this number reflects the heuristic’s behavior on synthetic data at least as much as it reflects a real “stated $\neq$ actual” phenomenon. This caveat must be stated explicitly in the thesis text, not presented as an empirical finding about real users.
- **The 7-day linking window is a heuristic choice**, not derived from the data. A different window would change the link count and, likely, the divergence number. A sensitivity check (e.g. re-running at 3, 14, and 30 days) is worth doing if this number is used quantitatively rather than only descriptively in the thesis.
- **Not yet incorporated**: recency weighting (older watches currently count equally with recent ones) and `reviews.csv` sentiment as a third signal. Both are left for a later iteration if the divergence signal needs refining for Phase 4.
- **Worth re-running once real interaction data becomes available** (e.g. from the bachelor students’ external application) as a real-world comparison point against this synthetic-data baseline.

7 Summary and Next Steps

Phase 3 delivers a verified, per-user behavioral divergence signal on the Movie domain: 10,000 JSON profiles, each pairing a stated-interest ranking with an actual-interest ranking and a single Jaccard divergence score, plus a row-level search→watch link table. This is the first concrete piece of the thesis’s actual contribution (as opposed to the Phase 0/1 harness it is built on top of, documented separately).

Phase 4, not yet built, is the direct consumer of this output: candidate query generation will retrieve a user’s semantically similar past queries (embedding-based retrieval over their query log), combine them with a compact rendering of this phase’s **divergence** block in an LLM prompt (following the GQR/RA-GQR generation pattern from the literature review), validate candidates against the currently loaded domain schema, rank them along a query-similarity vs. behavior-similarity axis, and expose the result as a `POST /recommend` API endpoint returning three suggested queries. Full detail on Phase 4 onward is tracked in the project roadmap document, *Thesis Project Plan*.